GATE & ISRO questions :

Q1)
```
Void main() {
    int a[2][3] = {5,10, 15,20,25, 30};
    int (*ptr)[2][3] = & array;
    printf("%d", ***ptr);
    printf("%d", ***(ptr+1));
    printf("%d", *(*(*ptr+1)+2));
}
```

output : 5

random value {garbage} {address of / error}

30

Q2)
```
main(){
    char *c[] = {"Facebook", "Amazon", "Google", "Netflix"};
    char **cp[] = {c+3, c+2, c+1};
    char ***cpp = cp;

    Pf("%s", **(++cpp));
    Pf("%s", *--*++cpp+3);
    Pf("%s", *cpp[-2]+3);
    Pf("%s", cpp[-1][-1]+1);
```

output : google

ebook

flix

mazon

— Tree —

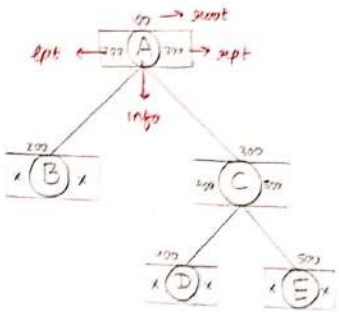
Tree Traversals :

Preorder	Inorder	Postorder
(Root → left → Right)	(Left → Root → Right)	(Left → Right → Root)

Structure :

```
struct node {
    char info;
    struct node * lpt;
    struct node * rpt; };
    struct node * root;
}
```

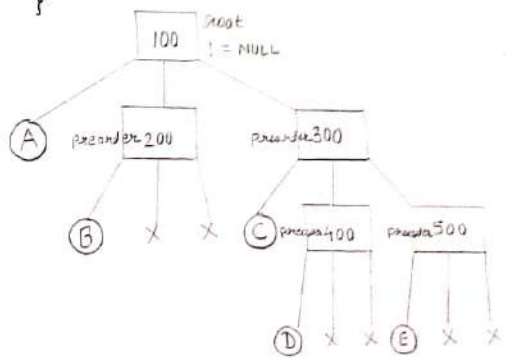
root node / initial like 'first' in LL



PREORDER :

```
void pre_order (struct node *root){
    if (root != NULL){
        printf("%c", root->info);
        Pre-order (root->lpt);
        pre-order (root->rpt);
    }
}
```

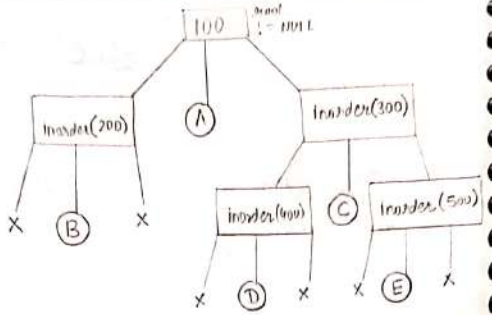
ABCDE



Scanned with OKEN Scanner

INORDER :

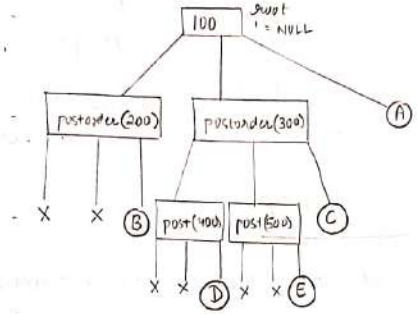
```
void in_order (struct node *root){
    if (root != NULL){
        in_order (root → lpt);
        printf ("%c", root → info);
        in_order (root → rpt);
    }
}
```



BADCE

POST ORDER :

```
void post_order (struct node * root){
    if (root != NULL){
        post-order (root → lpt);
        post-order (root → rpt);
        printf ("%c", root → info);
    }
}
```



BDECA

8) A computer scientist named Gautam creates a new traversal scheme with the name CSEORDER().

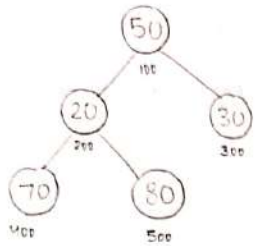
The nodes of the scheme are traversed in following order

Right → left → root → left → root → right

i) Write the program.

Scanned with OKEN Scanner

Creating a BINARY SEARCH TREE :




```
Struct node {
        Struct node * left ;
        int data ;
        Struct node * right ;
        };

Struct node * root , * ptr ;
Struct node * create ();
void display ( Struct node * ) ;
void construct ();

void main () {
        construct ();
        display (root);
        }

void construct () {
        root = create () ;
        root → data = 50;
        root → left = create();
        root → right = create();
        root → left → data = 20;
        root → right → data = 30;
        root → left → left = create();
        root → left → left → data = 70;
        root → left → right = create();
        root → left → right → data = 80;
```

```
Struct node * create () {
        ptr = (struct node *) malloc (sizeof(sn));
        ptr → left = ptr → right = NULL;
        return ptr;
    }

void display (struct node * root) {
        if (root != NULL) {
                display (root → left) ;
                display (root → right);
                printf("%d", root → info);
                }
        }
```

PG) WAP to count no. of leaf nodes in BST.

```
void count ( Struct node * root ) {
        if ( root == NULL )
                return ;
        int count = 0;
        if (root → left == NULL) && ( root → right == NULL) {
                count ++ ;
                }
        count (root → left) ;
        count (root → right);
    }
```

PG) WAP to get height of a Binary Tree.

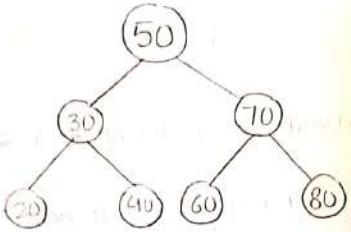
```
int height ( Struct node * root ) {
        int lheight = 0, rheight = 0;
        if (root == NULL) {
                return 0 ;
                }
        lheight = height (root → left) ;
        rheight = height (root → right) ;
        if (lheight > rheight) {
                return (lheight +1) ; }
        else {
                return (rheight +1) ;
                }
    }
```

Scanned with OKEN Scanner

P6) WAP to find the sum of all nodes of a tree.

```
int sum( Struct node * ptr, sum1 = 0) {
        if (ptr != NULL) {
                sum1 + = ptr → data;
                sum1 + = sum(ptr → left, sum1) + sum(ptr → right, sum1);
                }
        return sum1;
}
```

Creation & Searching in BST




```
Struct node {
        int info;
        Struct node* left;
        Struct node* right;
        };
Struct node * root = NULL;

Struct node * Insert (Struct node * node, int key) {
        if (node == NULL)
                return newnode (key);
        if (node → info > key) {
                node → left = Insert (node → left, key);
                }
        if (node → info < key) {
                node → right = Insert (node → right, key);
                }
        return node;
        }
```

```
Struct node * newnode (int item) {
        Struct node * temp;
        temp = (Struct node*)malloc (sizeof (Struct node));
        temp → left = temp → right = NULL;
        return temp;
        }

void inorder (Struct node * root) {
        if (root != NULL) {
                Inorder (root → left);
                printf ("%d", root → info);
                inorder (root → right);
                }
}

Struct node * Search (Struct node *root, int x) {
        if (root == NULL || root → info = x)
                return root;
        if (root → info > x)
                return Search (root → left, x);
        else
                return Search (root → right, x);
}

int main () {
        root = insert (root, 50);
        insert (root, 30);
        insert (root, 20);
        insert (root, 70);
        insert (root, 60)
        insert (root, 80);
}
```

Scanned with OKEN Scanner

Dry run:

I) I (X, 50)

X	50	X

root = 100

II) I (100, 30)

100 ! = NULL

100 → key = 50 > 30

100 → left = I (X, 30)

X	30	X

200

III) I (100, 20)

100 ! = NULL

100 → info = 50 > 20

100 → left = Insert (200, 20)

now; 200 ! = NULL

200 → info = 30 > 20

200 → left = I (200 → left, 20)

X	20	X

300

IV) I (100, 70)

100 ! = NULL

100 → info = 50 < 70

100 → right = I (100 → right, 70)

X	70	X

400

V) I (100, 60)

100 ! = NULL

100 → info = 50 < 70

100 → right = I (100 → right, 60)
400

now: 400 ! = NULL

400 → info = 70 > 60

400 → left = I (400 → left, 60)

X	60	X

500

VI) I (100, 80)

100 ! = NULL

100 → info = 50 < 80;

100 → right = I (400, 80)

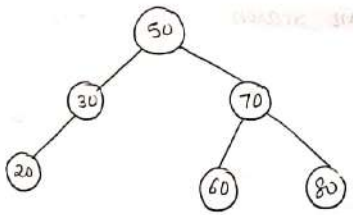
now; 400 ! = NULL

400 → info = 70 < 80

400 → right = I (400 → right 80)

X	80	X

formed tree:

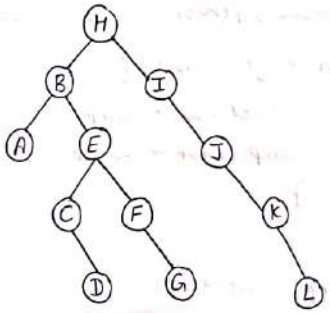


PG) WAP to find minimum in BST/ maximum in BST

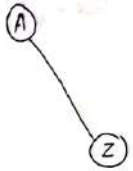
```
struct node * findmin (struct node * root) {
        if (root == NULL)
                return NULL;

minimum value node  { if (root → left ! = NULL)
                              return findmin (root → left);

maximum value node  { if (root → right ! = NULL)
                              return findmax (root → right);

        return root;
}
```

Q) Create BST:

(a) H I J B A E C F D G K L

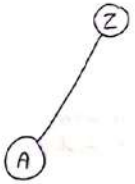


(b) A to Z



(Right skewed)

(c) Z to A



(left skewed)

Scanned with OKEN Scanner

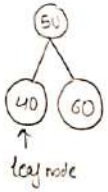
Deletion in BST :

```
Struct node *delete (Struct node * root, int x){
```

Traversal to find the node

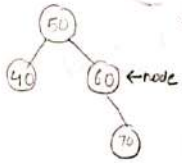
```
    if (root == NULL) {
            return NULL; }
    if (root -> info < x ) {
            root -> right = delete (root -> right, x); }
    else if (x < root -> info) {
            root -> left = delete (root -> left, x); }
```

I) leaf node



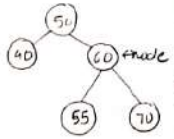

```
    else {
        if (root -> left == NULL && root -> right == NULL){
                free (root);
                return NULL;
                }
```

II) node with single child




```
        else if (root -> left == NULL ) || (root -> right == NULL){
            if (root -> left == NULL) {
                Struct node * temp;
                temp = root -> right;
                }
            else if {
                temp = root -> left;
                free (root);
                return temp;
                }
```

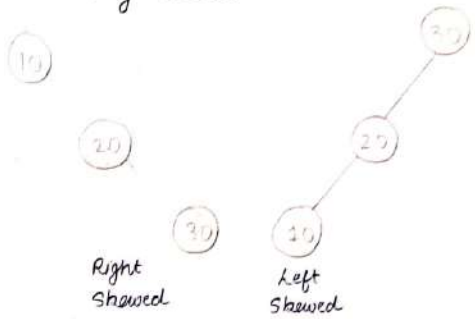
III) node with 2 child




```
    else {
        Struct node * Y = findmin (root -> right);
        root -> info = Y -> info;
        root -> right = delete (root -> right, x);
    }
```

AVL Tree

→ Self Balancing Trees



$T(C) = O(H)$
$= O(N)$

→ In AVL the time complexity i.e

Bset	Avg	Worst

→ $O(\log N)$

Balance factor

BF → Height of Left subtree − Height of right subtree ; $(AV)BF = H_L - H_R$

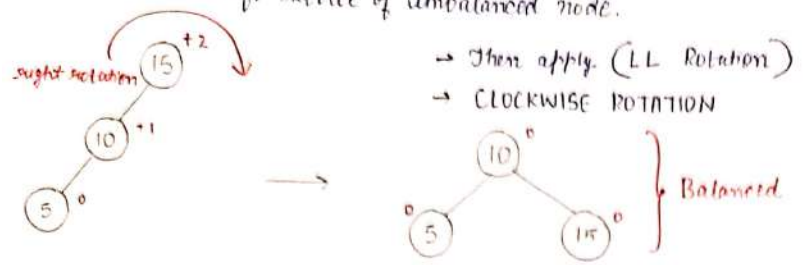
→ In AVL tree allowed BFs are $\{-1, 0, 1\}$

→ When the value of balance factor is
- -1 : Right heavy
- $+1$: Left heavy

- Right heavy : Left Rotation or anticlockwise
- Left heavy : Right rotation or clockwise

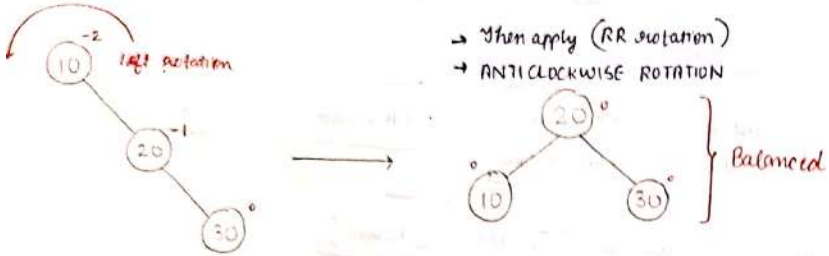
Scanned with OKEN Scanner

1) **LL Rotation** : when a new node is inserted as the left child of left subtree of imbalanced node.



→ Then apply (LL Rotation)
→ CLOCKWISE ROTATION

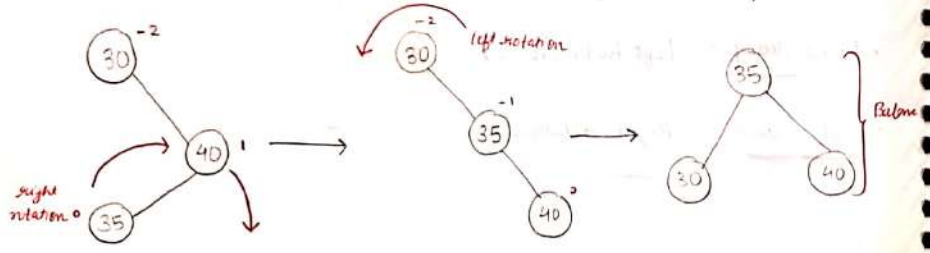
2) **RR Rotation** : when a new node is inserted as the Right child of right subtree of imbalanced node.

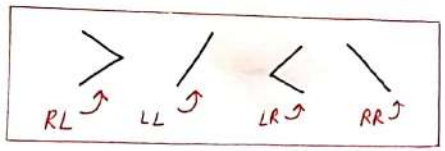


→ Then apply (RR rotation)
→ ANTICLOCKWISE ROTATION

3) **RL Rotation** : when a new node is inserted as the left child of the right subtree of imbalanced node.

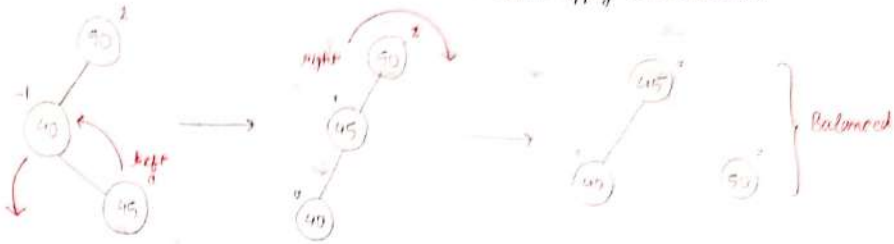
→ Then apply (RL rotation)



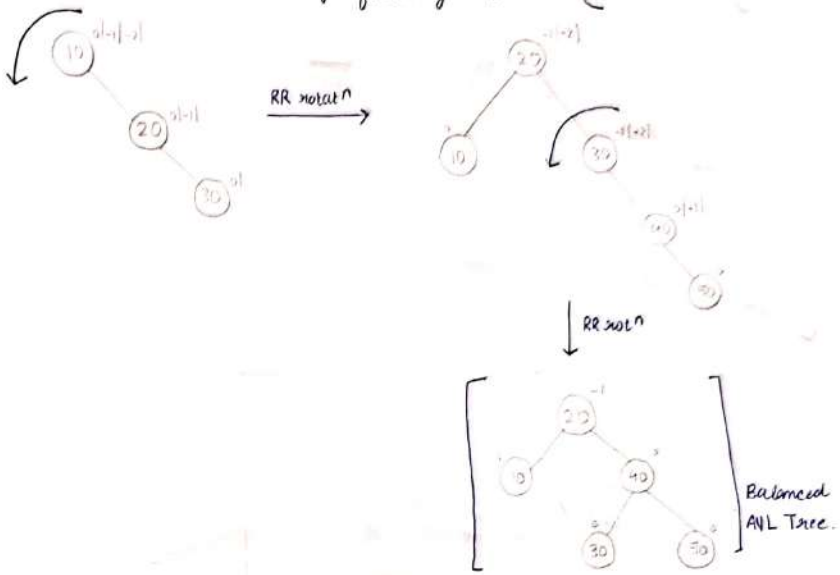


4) **LR Rotation** : If the new node is inserted as the right child of left subtree of imbalanced node is called LR rotation

→ Then apply LR rotation



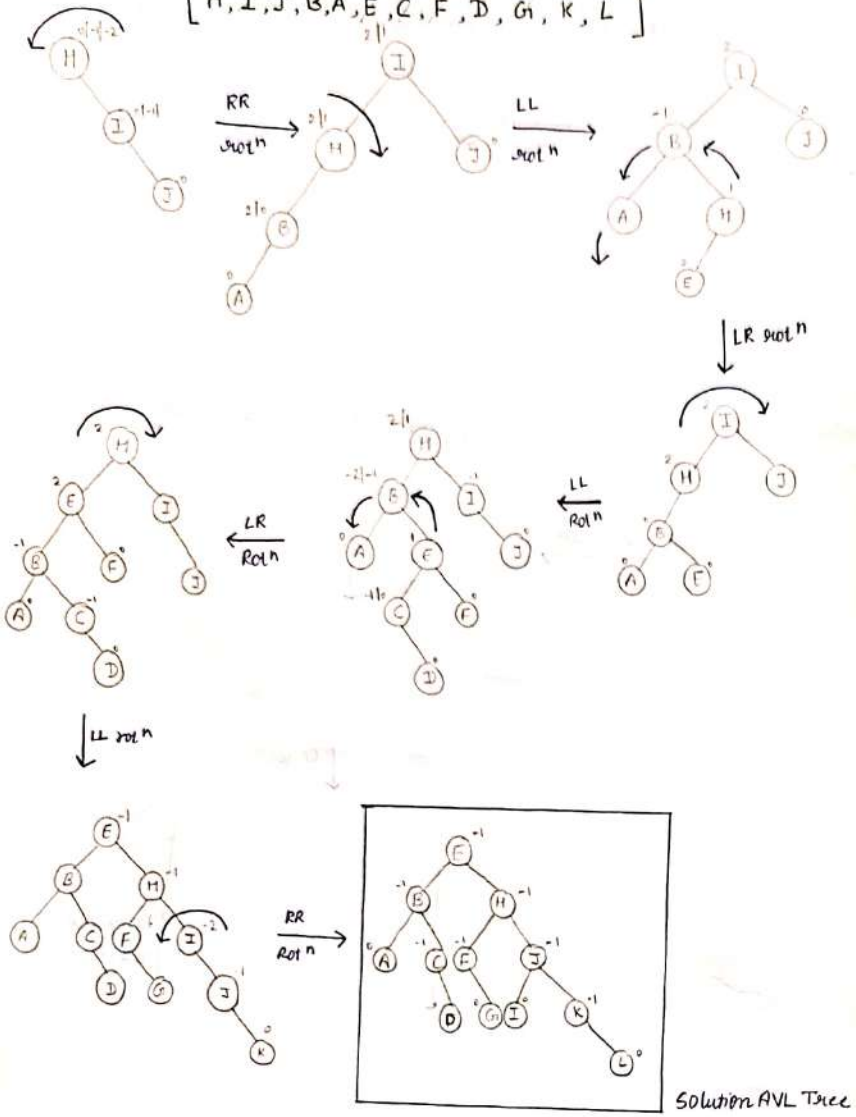
5) Construct an AVL tree by following sequence. (10 20 30 40 50)



Scanned with OKEN Scanner

Q) Construct an AVL tree for the following sequence.

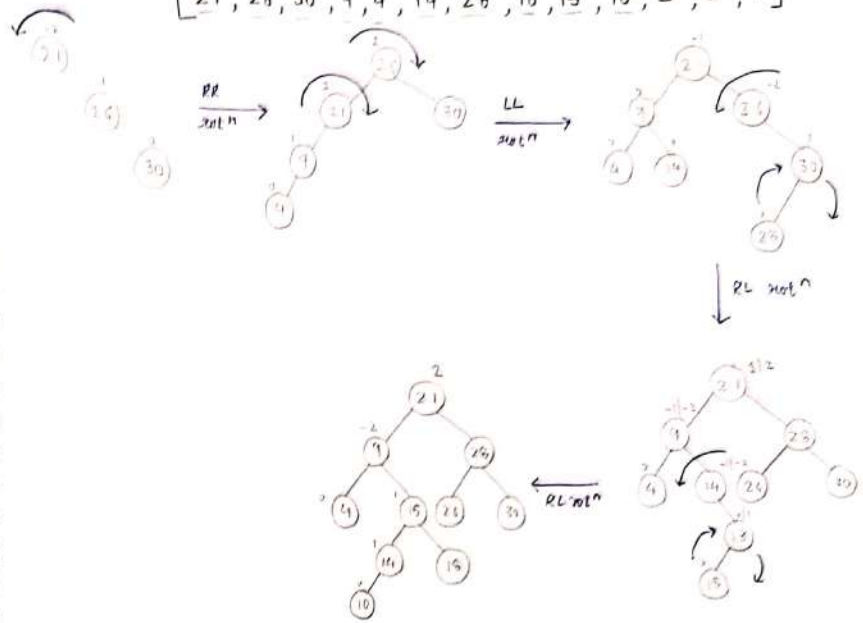
[H, I, J, B, A, E, C, F, D, G, K, L]



Solution AVL Tree

Q) Construct an AVL Tree for the given sequence:

[21, 26, 30, 9, 4, 14, 28, 18, 15, 10, 2, 3, 7]



Scanned with OKEN Scanner

B - Tree
B^+ - Tree } Complex DS

Red Black Tree

Splay Tree
2-3 Tree } Complex DS

→ B-Tree maintain its balance by ensuring that each node has minimum no. of so that the tree is always balanced. This balance guarantees that the complexity for insertion, deletion and searching is always $O(\log n)$

→ B-Trees are characterized by large no. of key that they can store in a single node which is why they are also known as large key trees.

→ Each node in as B-Tree can contain multiple keys which allows the trees to have a large branching factor and thus a shallower height.

→ This shallow height leads to less disk i/p o/p which result in faster search and insert operation.

→ B-Trees are particularly well suited for storage system that have slow and bulky data access such as hard disks, flash memory and CD ROMs.

→ B-Tree of order (M) will have the following properties :-

Maximum keys $= M-1$

Minimum keys $= \left\lceil \frac{M}{2} \right\rceil - 1$

Max^m children $= m$

Min^m children $= \left\lceil \frac{m}{2} \right\rceil$

→ All the keys are at the same level.

→ No. of children of a node = No. of keys + 1

→ All the keys are stored in increasing order.

Scanned with OKEN Scanner

→ B-Tree grows and shrinks from the root which is unlike BST.

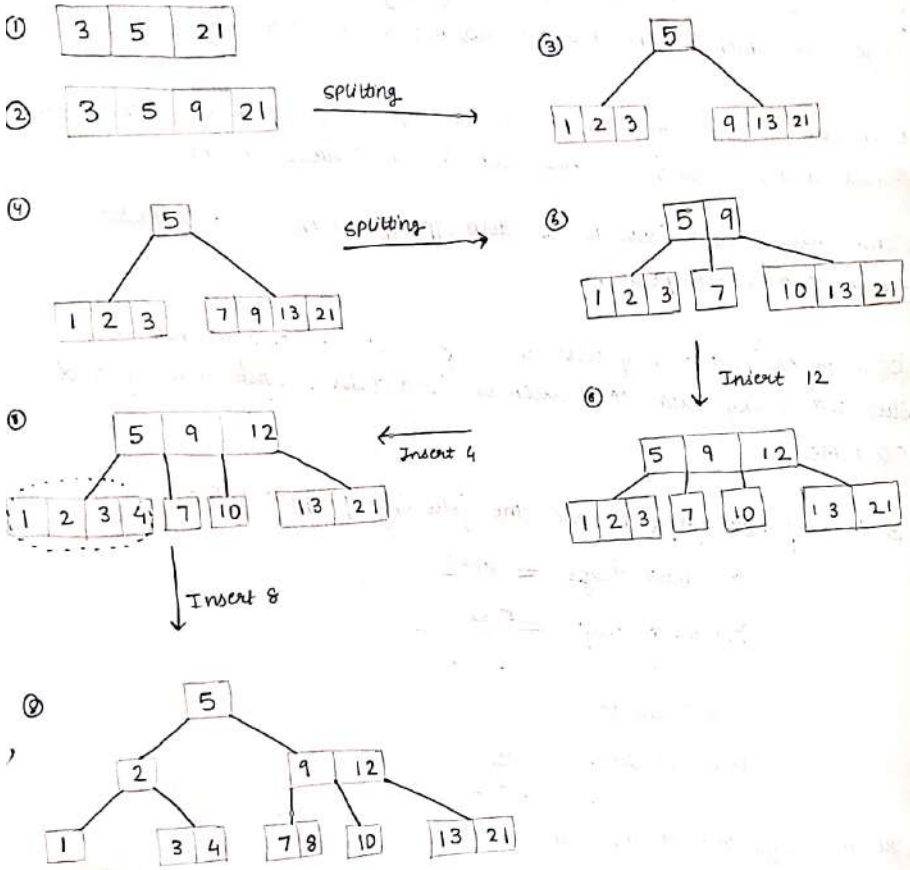
→ Insertion of a node always happens only at the leaf node.

Q) Construct a B-Tree of order 4 for following elements;

[5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8] (m=4)

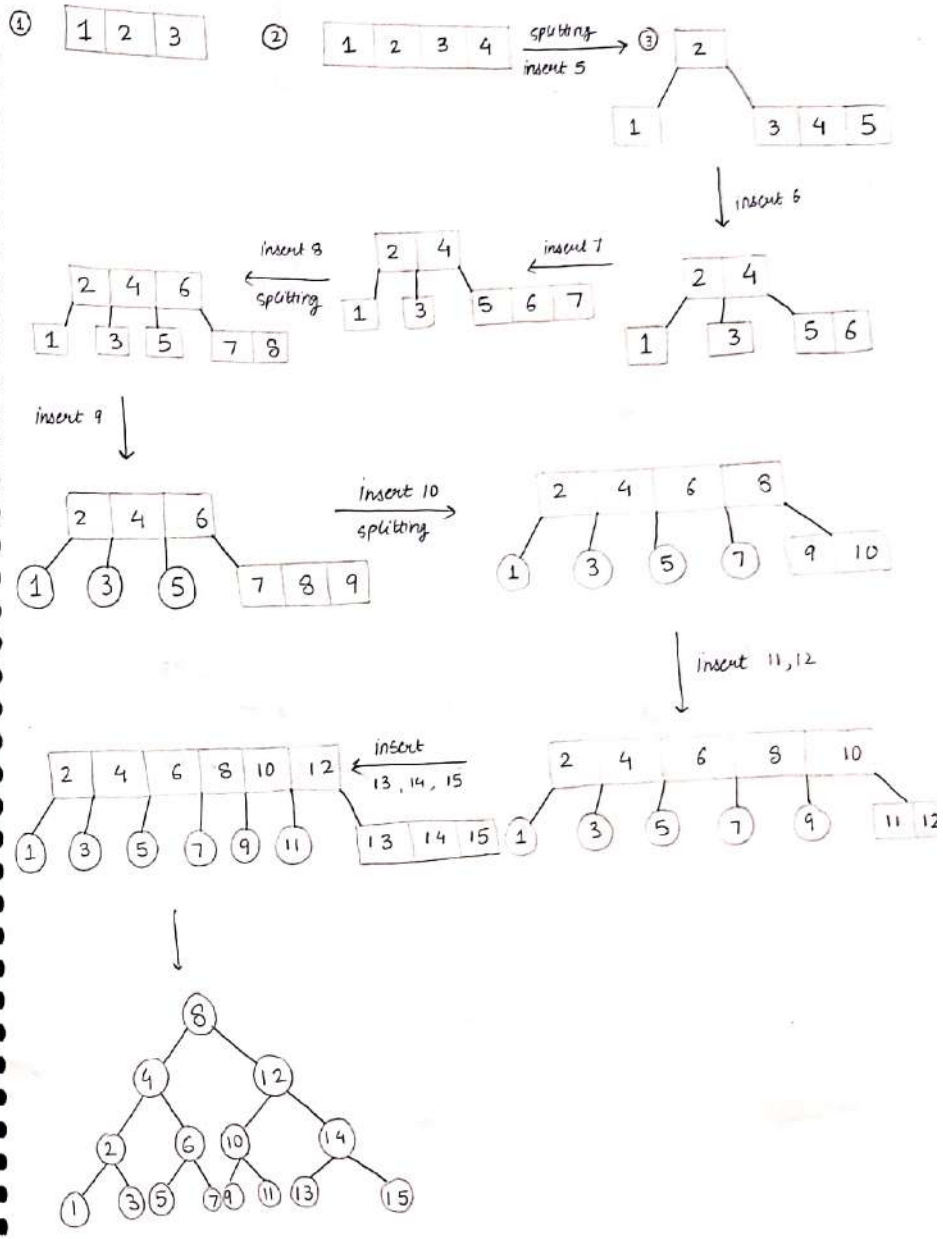
max keys = m - 1 = 3

min key = $\left\lceil \frac{4}{2} \right\rceil - 1 = 1$

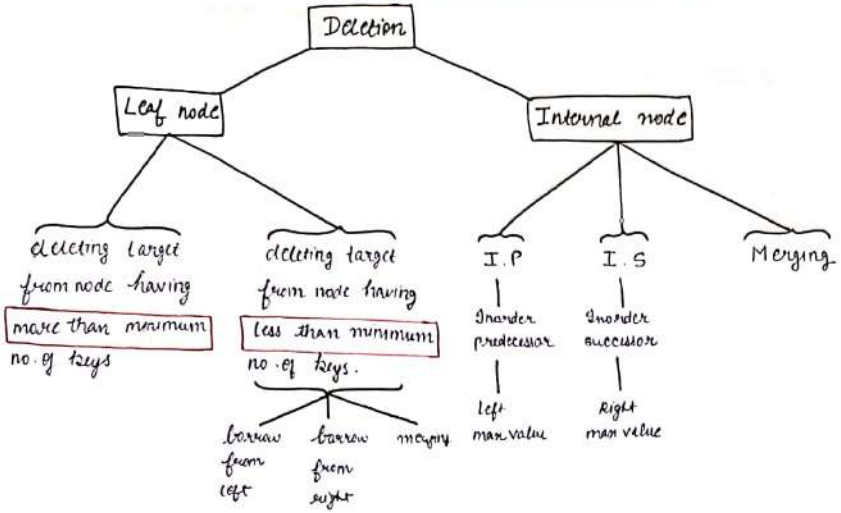


Q) Make a B-Tree from integer 1-15 of order m=3.

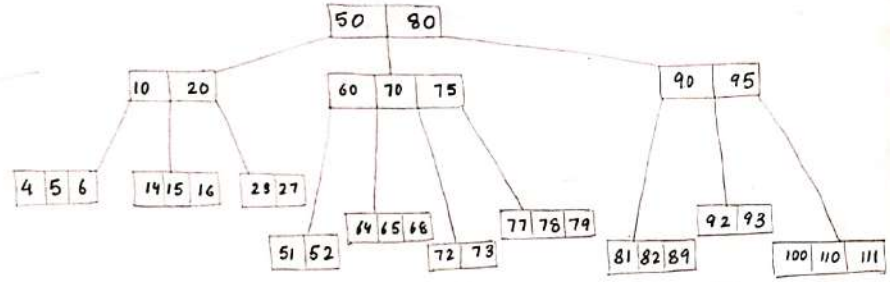
max key = 2 min key = 2



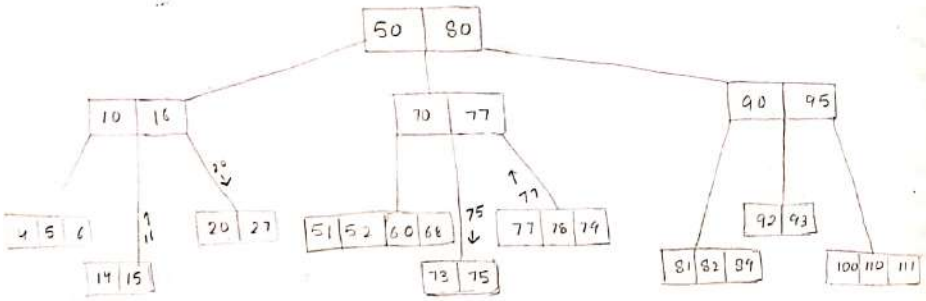
Scanned with OKEN Scanner



Q) Perform deletion in B-Tree

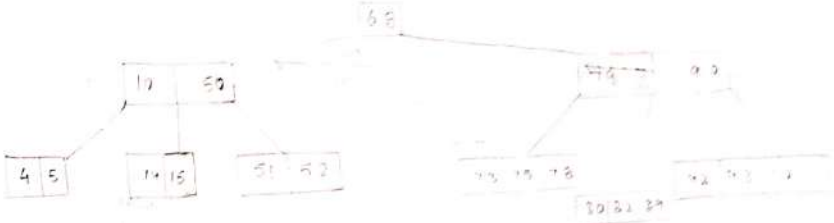


deletion order: 64, 23, 72, 65, 20, 70, 95, 77, 80, 100, 6, 27, 16, 60, 50

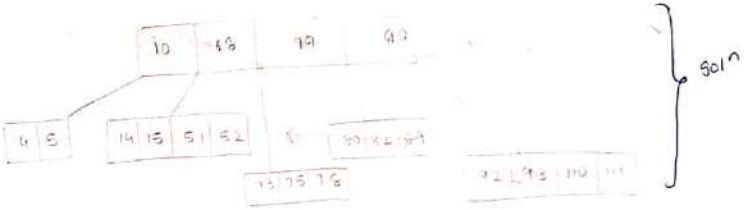


to delete 20 we'll merge 10 16 with 50 causing shift of 70 & 77

after deletion of 100, 99 will be left alone (our foresight) so we'll move to 99, making 73 75 78 shift



Deletion of 50 will cause merging of 10 68 79 90 and have the above



B+ Tree

→ Data is stored only in leaf nodes unlike B-tree where data is stored in both leaf and internal node.

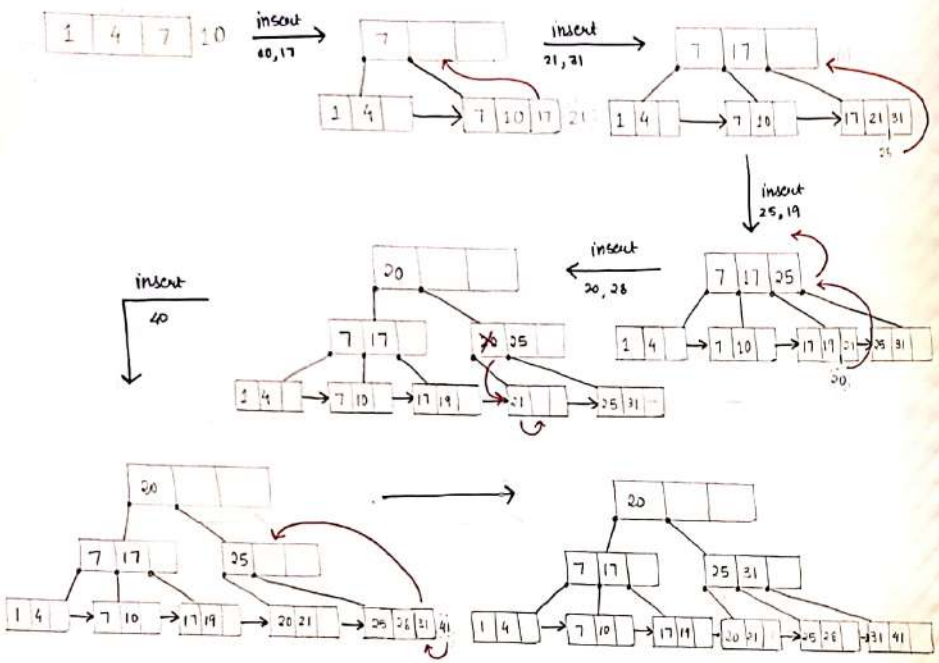
→ Internal nodes contain only the pointer or indexes to the actual data.

→ Data in leaf node are connected using linked list.

→ Indexes or the pointers cannot be repeated in root and internal nodes.

Q) Create a B+ tree of order 4 for the following given nodes:

[1, 4, 7, 10, 17, 21, 31, 25, 19, 20, 28, 42]



NOTE: m=4

max no. of keys = m-1 = 3

min no. of keys = $\left[\frac{m}{2}\right]-1 = 1$

max child = m = 4

min child = $\left[\frac{m}{2}\right] = 2$

Assumptions:

1) Right Biased tree splitting.

2) Left of an index will be strictly lesser

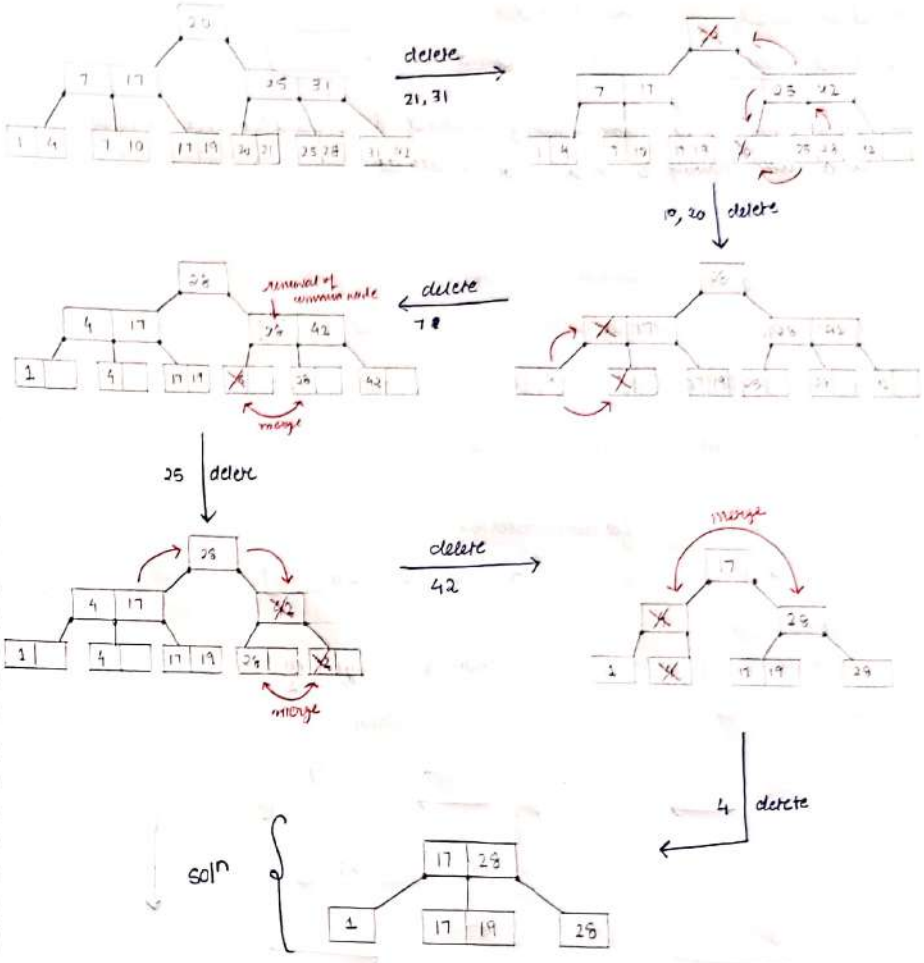
3) Right will be equal to or greater.

Deletion

Q) Delete from B+ tree in order [21, 31, 20, 10, 7, 25, 42, 4]

max children = 4 max keys = m-1 = 3

min children = $\left[\frac{m}{2}\right] = 2$ min keys = $\left[\frac{m}{2}\right]-1 = 1$



soln

2-3 Trees

→ Like AVL Trees, Red-Black trees, B-tree 2-3 Tree is also a high balanced tree.

→ The time complexity for insertion, searching, deletion is $O(\log n)$.

→ It is called as B-tree of order 3.

→ A node having 2 children are called as (2-nodes).

→ A 2-node is a node having 2 child & 1 data set and 3-node is a node having 3 child and 2 data set.

Properties :

1) Data is stored in a sorted manner.
2) All the leaf nodes are at the same rules level.
3) Each node can be a leaf-node, a 2-node or a 3-node.
4) Insertion is always done at leaf node.

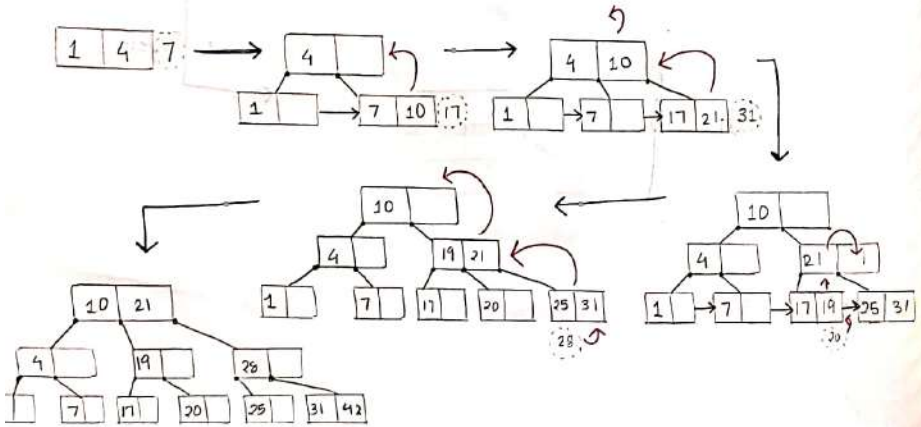
Q) Create a 2-3 Tree of following nodes:

[1, 4, 7, 10, 17, 21, 31, 25, 19, 20, 28, 42]

$m = 3$

max no. of key $\Rightarrow m - 1 = 2$ min no. of keys $\rightarrow \frac{m}{2} - 1 = 1$

max children $\rightarrow m = 3$ min children $= (m/2) = 2$



2-3-4 Trees

→ 2-3-4 tree is a B-tree of order 4.

$m = 4$

max key $= 3$

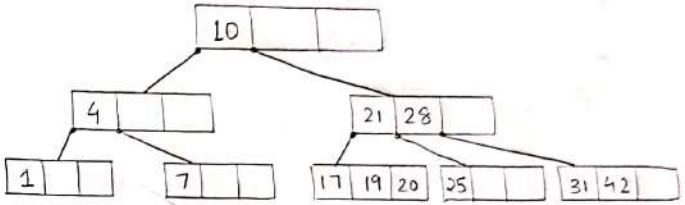
min key $= \left\lceil \frac{m}{2} \right\rceil - 1 = 1$

→ 2-3-4 Tree can have leaf node, a 2 node, a 3 node and a 4 node

→ Rest all features / complexities / properties are same as B-Tree.

Q) Construct 2-3-4 Tree for the following:

[1, 4, 7, 10, 17, 21, 31, 25, 19, 20, 28, 42]



Scanned with OKEN Scanner

SPLAY TREE

→ It's a self balancing BST, which means that a free structure is existing dynamically based on the access or inserted elements, in other words the tree automatically reorganises itself so that the frequently accessed or inserted elements becomes closer to the root node.

→ It has a simple and efficient implementation that allow it to perform search, insertion, deletion and updation in $O(\log n)$

→ The basic idea behind a SPLAY TREE to bring the most recently accessed or inserted element to the root of the tree by performing a sequence of tree rotations called as Splaying.

→ Splay trees are highly efficient in practise due to their self adjusting nature which reduces the ~~excess~~ access time for frequently accessed elements, this makes it a good choice for applications that requires fast and dynamic DS such as,
- Caching system
- Data compression
- Network routing algorithm

→ It is the fastest type of BST for various practical applications it is used to windows and NT and GCC compiler.

→ **Disadvantages:**

→ The main disadvantage is that it does not guarentee a balanced tree struc. which may lead to performance degradation in worst case scenerios, they are roughly balanced, sometimes they are linear and the time comp. becomes $O(n)$.

Scanned with OKEN Scanner

→ Splay trees are not suitable for application that requires guaranteed worst case performance like critical safety system.

→ **NOTATIONS in SPLAY TREES :** (Zig → Right Zag → left)

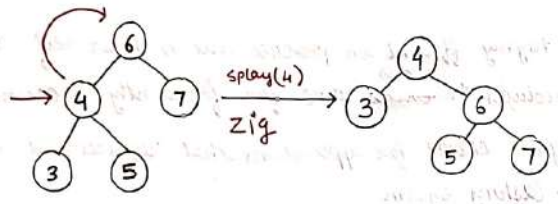
→ There are 6 rotations in splay trees :

① Zig ④ zag - zag

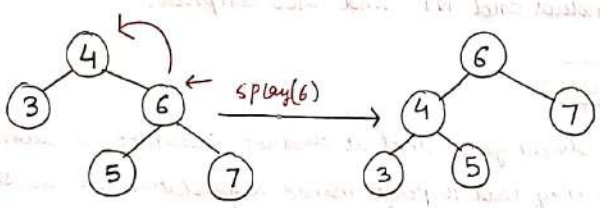
② zag ⑤ zig - zag

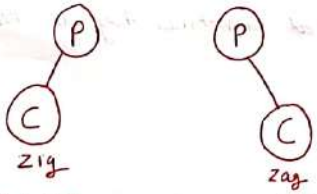
③ zig - zig ⑥ zag - zig

Zig : This rotⁿ results in nodes moving one posⁿ to the right from their root postⁿ it is similar to right rotⁿ in AVL tree.



Zag : During this rotⁿ node shift one posⁿ to the left from their current location, it is similar to left rotation of AVL tree.

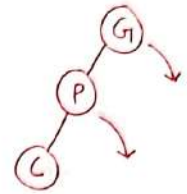




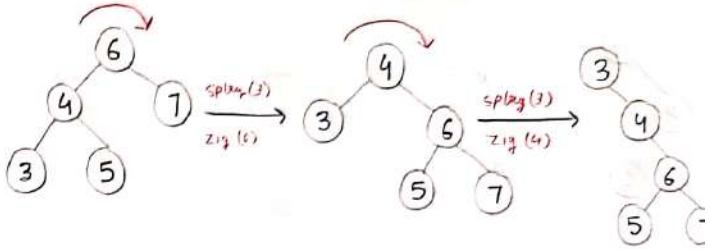
zig
rotate(P)

zag
rotate(P)

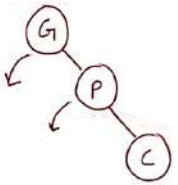
Zig - Zig : It's a double zig rotⁿ, the rotⁿ results in node shifting 2 posⁿ to the right from their current location



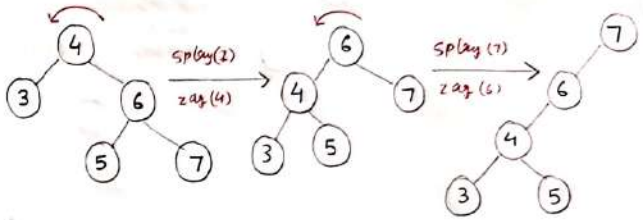
splay(C)
→ rotate(G) Right
→ rotate (P) Right



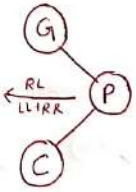
Zag - Zag : It's a double zag rotⁿ, the rotⁿ causes node to move two posⁿ to left from their current posⁿ.



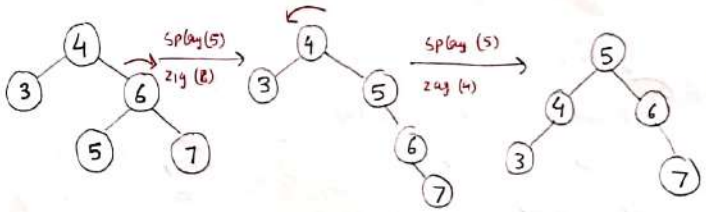
splay(C)
→ rotate (G) Left
→ rotate (P) Left



Zig - Zag : It's a combⁿ of zig rotation followed by a zag rotⁿ as a result of this rotⁿ node shift one posⁿ to right and then one postⁿ to left from current location.

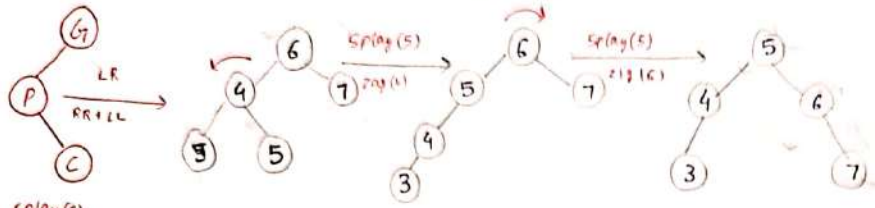


splay (C)
→ rotate (P) right
→ rotate (G) left



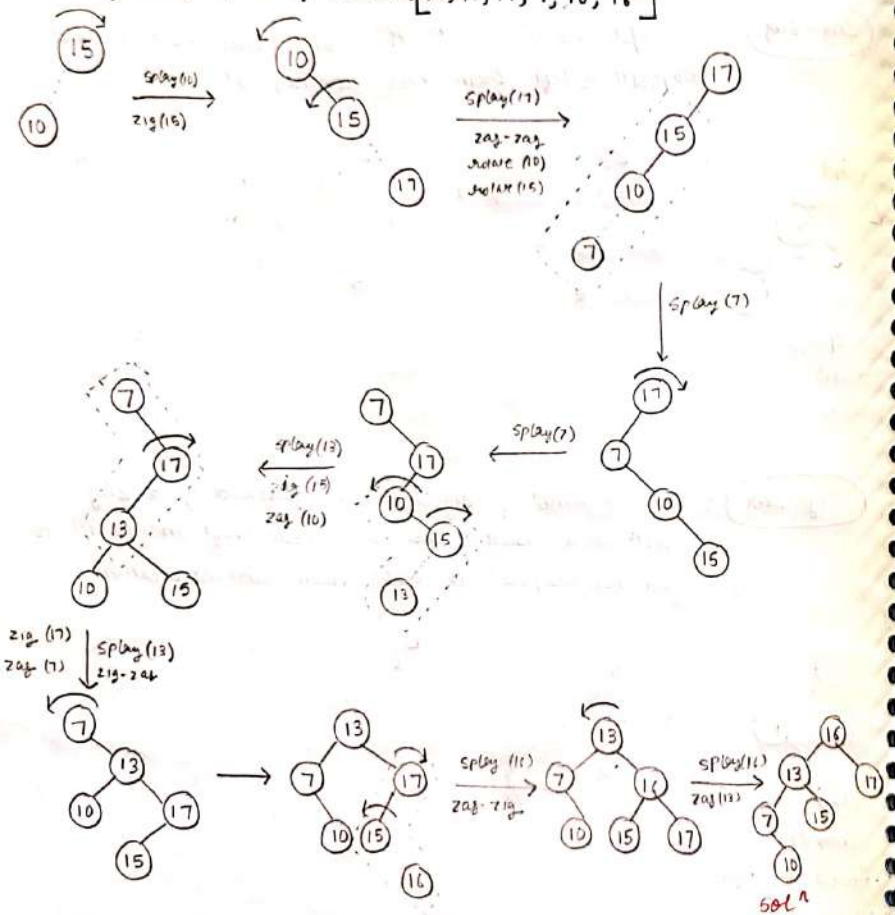
Scanned with OKEN Scanner

Zag-Zig: It's a comb^n of zag rot^n followed by zig rot^n, this result of node shift one position to the left followed by a shift of one post^n to right from their current post^n

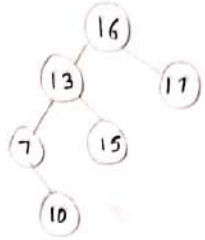


splay(c)
rotate (P) left
rotate (G) right

B) Create a splay tree for following elements: [15, 10, 17, 7, 13, 16]



Code algorithm:



Insert (14)

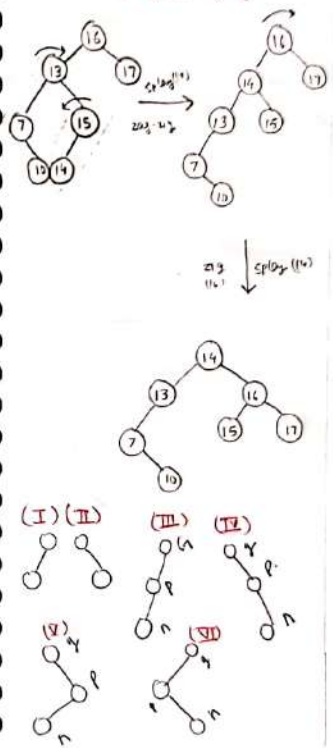
(I) Searching in BST to find post for (14).

(II) Insertion of (14)

(III) Splaying of (14).
splay(T, n)

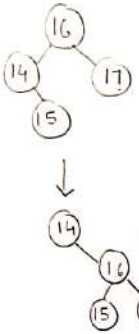
```
Insert (T, n) {
temp = T.root
y = NULL
while (temp != NULL) {
        y = temp.
        if (n -> data < temp -> data)
                temp = temp -> left
        else
                temp = temp -> right
}
n.parent = y
if (y == NULL)
        T.root = n
else if (n -> data < y -> data)
        y -> left = n
else
        y -> right = n
}
```

Splay (T, n) {

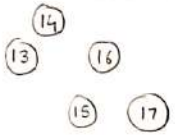



```
while (n->parent != NULL)
    if (n->parent == T->root)
        if (n == n->parent->left)
            right_rotation (T, n->parent)  (I)
        else
            left_rotation (T, n->parent)  (II)
    else
        p = n->parent
        g = p->parent

    if (n == p->left && p == g->left)
        right_rotation (T, g), right_rotation (T, p)  (III)
    else if (n == p->right && p == g->right)
        left_rotation (T, g), left_rotation (T, p)  (IV)
    else if (n == p->left && p == g->right)
        right_rotation (T, p), left_rotation (T, g)  (V)
    else
        left_rotation (T, p), right_rotation (T, g)  (VI)
```

Scanned with OKEN Scanner



```
right_rotation (T, X) {
        y = X → left
        X → left = y → right
        y → right = X
    return y
    }
```



```
left_rotation (T, X) {
        y = X → right
        X → right = y → left
        y → left = X
    return y
    }
```

DELETION in Splay tree :

→ There are 2 types of deletion in Splay tree.

i) Bottom – Up splaying

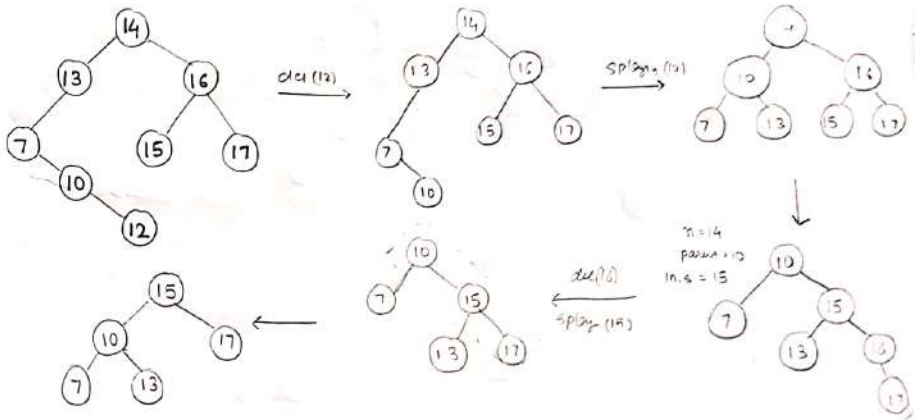
ii) Top – down splaying

→ In Bottom-up splaying first we delete the element from tree and then we perform splaying of parent of deleted node.

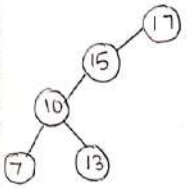
→ In Top-down splaying we first perform splaying on which the deletion is to be performed and then delete the node from tree.

→ Once the element is deled we perform Join operation.

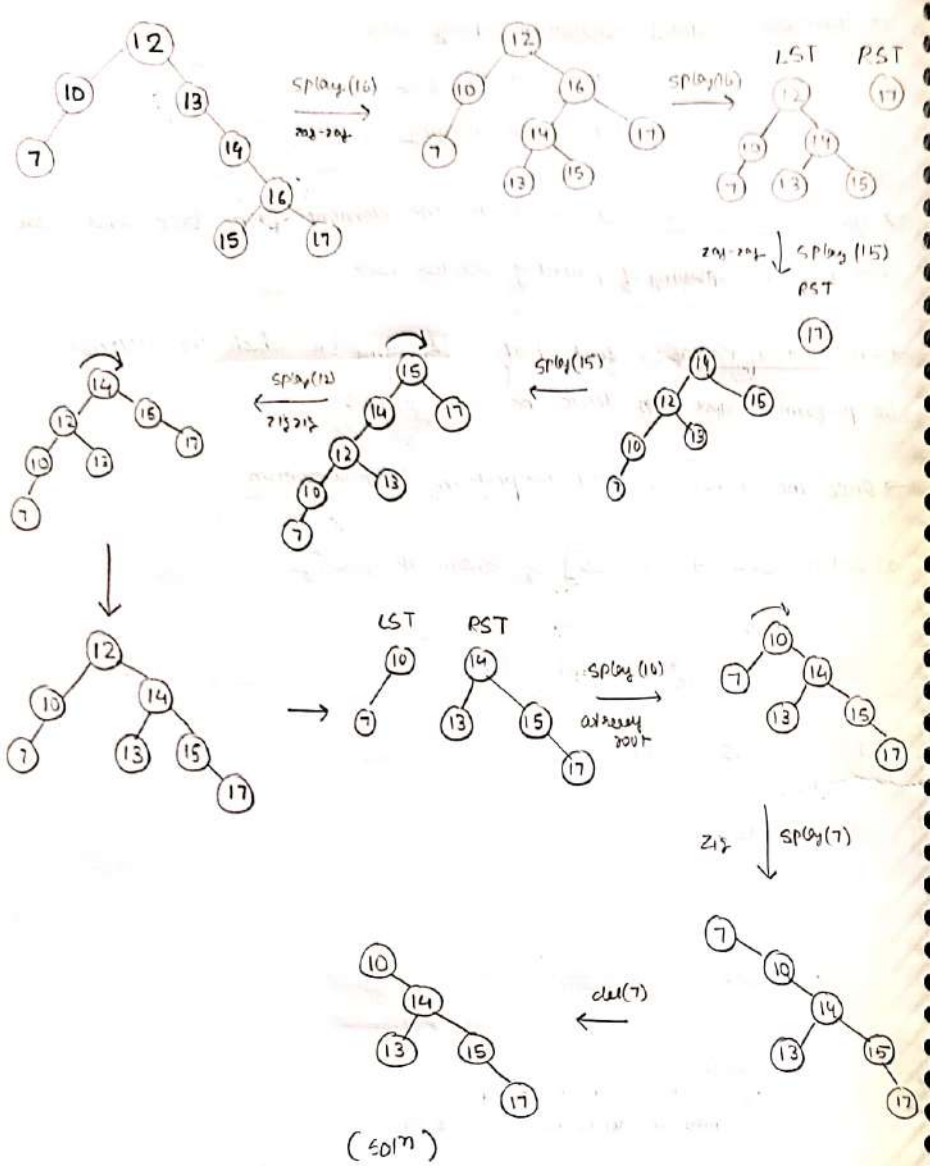
Q) Delete [12 14 16 20] by Bottom-up splaying



del(20)
after searching (20) which is not present
so splay the last not null node i.e 17.



Q) Delete the elements [16, 12, 7]



RED - BLACK TREE :

→ It is a self balancing tree.

→ Every node is either a black node or red node.

→ Root is always black in colour.

→ Every leaf node which is NIL is black.

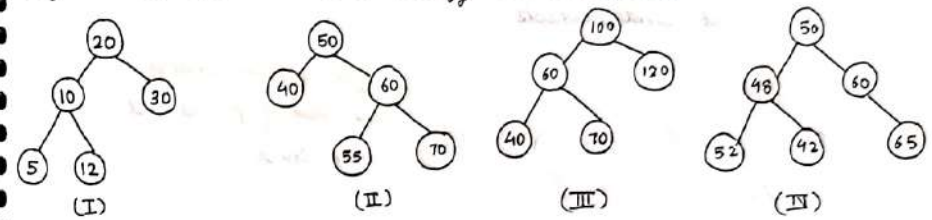
→ If a node is red in colour, then it's children are in black color.

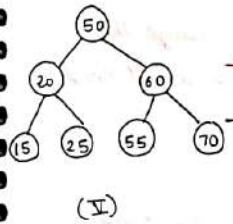
→ Every path from a node to any of its decendant NIL node have same no. of black node.

→ Red-black tree are roughly height balanced, but AVL is strictly height balanced tree.

→ AVL is subset of Red-black tree.

Q) given the tree we have to identify Red-Black tree.





→ A perfect binary tree with all BLACK NODE is a RED BLACK tree

→ a perfect BT is a BT where all nodes have 2 child nodes and all leaf nodes are on same stage / level.

Scanned with OKEN Scanner

Steps for insertion in Red-Black tree :

1) If tree is empty the create a node with color Black

2) If tree is not empty then create a new node / leaf node with color Red

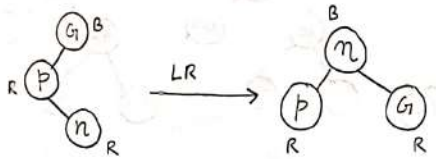
3) If the parent of new node is ~~red~~ black then exit.

4) If parent of new node is red then check the color of parent sibling of new-node.

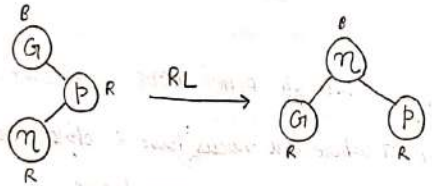
[4.1] If the color is black or nil then do suitable rotations & recolour.

[4.2] If the color is red then recolour parent and parent sibling and also check if parent's parent of new node is not the root node then recolor it and recheck.

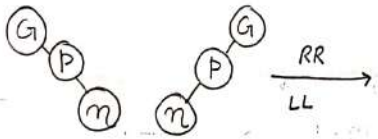
(4.1)



after rotation change the color of parent and right child

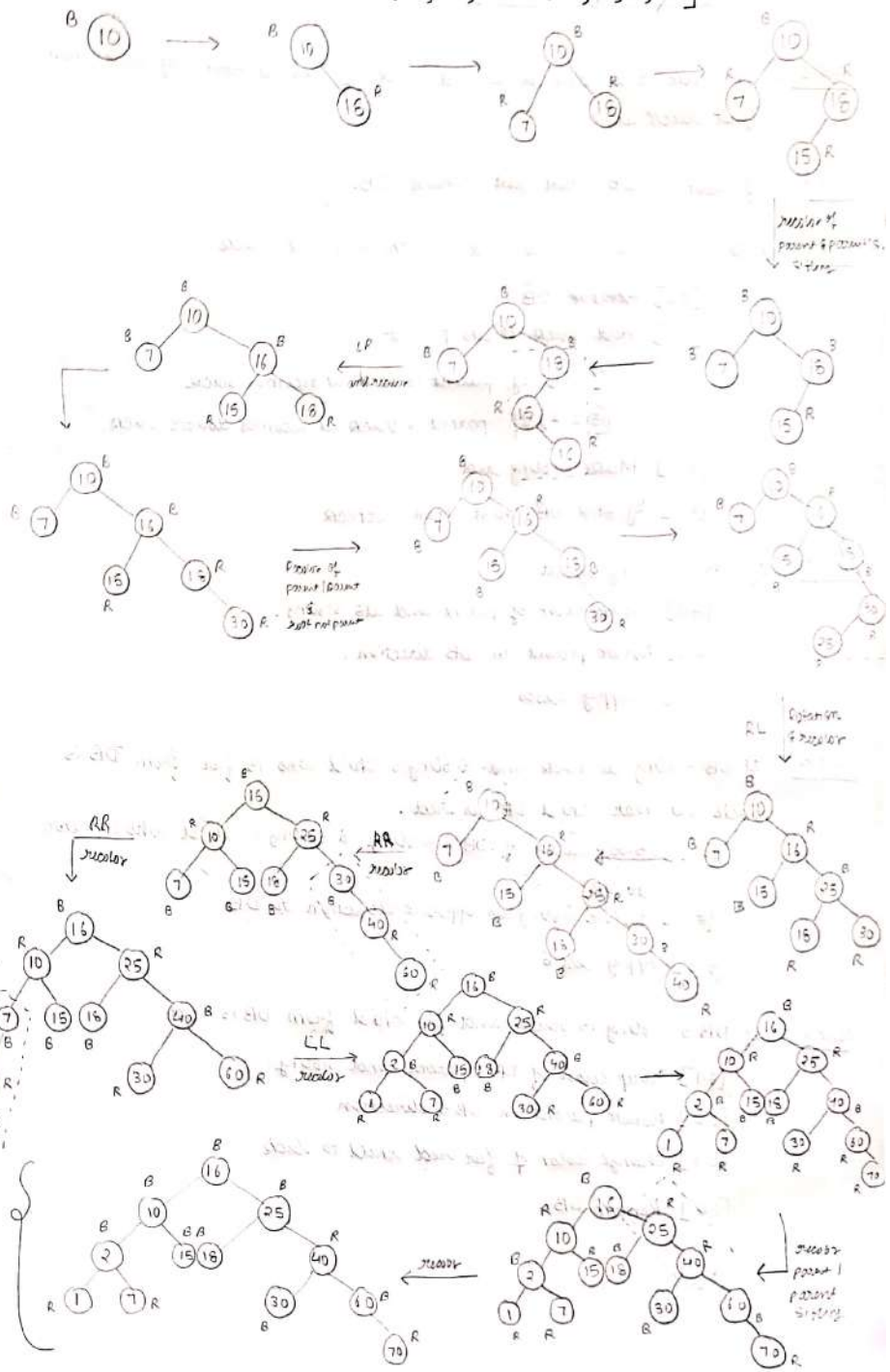


after rotation change the color of parent and left child

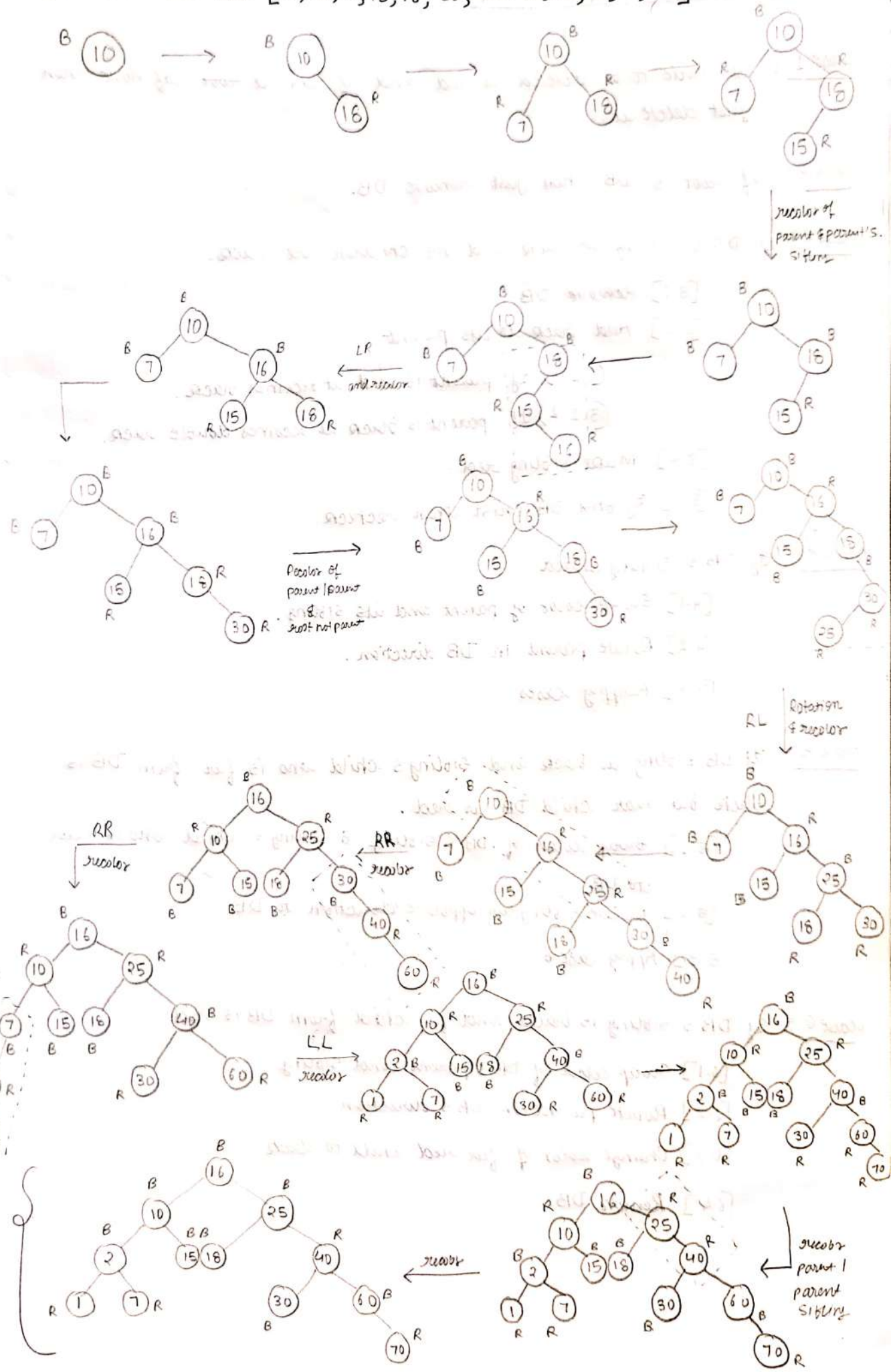


after rotation change the color of parent and parent's parent.

Q) Create Red-Black tree [10, 18, 7, 15, 16, 30, 25, 40, 60, 2, 1, 70]



Q) Create Red-Black tree [10, 18, 7, 15, 16, 30, 25, 40, 60, 2, 1, 70]



Steps to deletion in RB Tree:

case 1: If node to be deleted is red and if its a root leaf node then just delete it.

case 2: If root is DB then just remove DB.

case 3: If DB's sibling is black and its children are black.

[3.1] Remove DB

[3.2] Add black to its parent

(3.2.1) If parent is red it becomes black.

(3.2.2) If parent is black it becomes double black.

[3.3] Make sibling red.

[3.4] If still DB exist then recheck.

case 4: If DB's sibling is red

[4.1] Swap color of parent and its sibling.

[4.2] Rotate parent in DB direction.

[4.3] Reapply cases

case 5: If DB sibling is black and sibling's child who is far from DB is black but near child DB is red.

[5.1] Swap color of DB's sibling & sibling's child who is near to DB.

[5.2] Rotate sibling in opposite direction to DB

[5.3] Apply case 6

case 6: If DB's sibling is black and far child from DB is red.

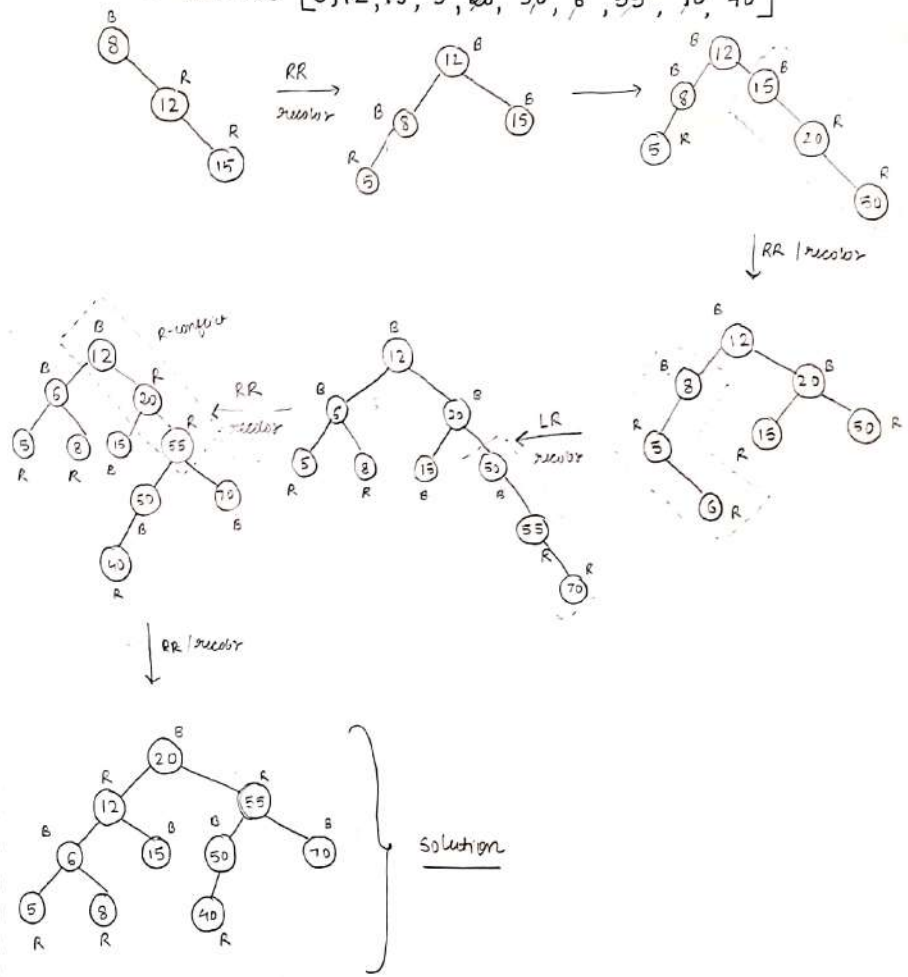
[6.1] Swap color of DB's parent and sibling.

[6.2] Rotate parent in DB's direction.

[6.3] Change color of far red child to black

[6.4] Remove DB.

Q) Create Red Black tree [8, 12, 15, 5, 20, 50, 6, 55, 70, 40]



Solution